Identifying DVD-ripped episodes for Plex

This guide covers the **focused workflow** for the real problem: “I have a folder of episode files ripped from DVDs - they’re out of order, some are extended cuts, and the on-screen video never shows the episode title. Which file is which episode, so I can name them for Plex?”

It uses one small script - `identify_dvd_episodes.py` - and a reference database built from the show’s subtitles. You do **not** need a microphone, and you do **not** transcribe the whole video.

How it works (and why)

For each video file the tool:

1. **Samples a few short audio clips** from spread-out timestamps (default **10%, 30%, 50%, 70%, 90%** of the runtime, ~12 s each).
2. **Transcribes each clip** using the Vosk offline speech-to-text engine.
3. **Phonetically matches the transcribed dialogue** against the subtitle-based reference database using Double Metaphone encoding and shingle hashing.
4. **Votes** across the samples. Requiring several spread-out clips to agree makes the result robust to “previously on…” recaps, cold opens, ad-break gaps, and extended-cut inserts that could fool a single sample.
5. **Falls back to fuzzy (LCS) phonetic matching** when exact shingle matching is weak - this tolerates dropped, mis-heard, or inserted words from noisy speech-to-text output.
6. **Optionally sanity-checks the runtime** against expected episode lengths.
7. Writes a `filename -> episode` **map** (CSV + JSON) and flags anything low-confidence for **manual review**.

Parallel processing

When identifying many files, use the `--workers` flag to process multiple files in parallel:

```
python identify_dvd_episodes.py --dir /path/to/dvd_rips --workers 4
```

Each worker handles one file independently, so on a multi-core machine this can cut total processing time significantly.

Step 1 - Build the reference database from subtitles

You only do this once per series. Download the SRT files for the show, then fingerprint them.

Get the SRT files

- **Automatic** (OpenSubtitles, configured in `config.json`):
  `bash`
  `python create_fingerprint.py --show "Matlock (1986)" --season 1`
- **Manual**: drop `.srt` files into a folder. Name them so season/episode can be detected, e.g.:
  `Matlock (1986) - S01E01 - Diary of a Perfect Murder.srt`
  `Matlock (1986) - S01E03 - The Judge.srt`
  `Matlock (1986) - S01E04 - The Stripper.srt`

Fingerprint the subtitles (phonetic reference)

```
python create_fingerprint.py --dir /path/to/subtitles
```

Check what's in the DB anytime:

```
python create_fingerprint.py --list
```

Step 2 - Batch-identify your DVD rips

Point the tool at the folder of unknown files:

```
python identify_dvd_episodes.py --dir /path/to/dvd_rips \
    --csv episode_map.csv --json episode_map.json
```

For faster processing on multi-core machines:

```
python identify_dvd_episodes.py --dir /path/to/dvd_rips \
    --csv episode_map.csv --json episode_map.json --workers 4
```

Example console output:

```
================================================================================
  DVD EPISODE IDENTIFICATION
================================================================================
  reference DB : fingerprints.db
  files        : 12
  sample points: 10%, 30%, 50%, 70%, 90% (12s each)
  workers      : 4

>>> Disc1_Title3.mkv
    => S01E04  Matlock (1986)  [phonetic, conf 35%, 4/5 samples]  ✓

>>> Disc2_Title1.mkv
    => S01E07  Matlock (1986)  [phonetic, conf 22%, 3/5 samples]  ⚠ REVIEW
       note: samples disagree (3/5)
...
================================================================================
  SUMMARY
================================================================================
  identified confidently : 10/12
    ✓ Disc1_Title3.mkv        -> S01E04  (phonetic, 35%)
    ...
  needs manual review    : 2
    ⚠ Disc2_Title1.mkv        -> S01E07  (samples disagree (3/5))
```

The **CSV / JSON** contains one row per file:

file-name	epis-ode_id	title	confid-ence	agree-ment	meth-od	dura-tion_s	needs_ review	notes
Dis-c1_Title 3.mkv	S01E04	Matlock (1986)	0.35	4/5	phonet-ic	2880.0	False	
Dis-c2_Title 1.mkv	S01E07	Matlock (1986)	0.22	3/5	phonet-ic	3120.0	True	sample s dis-agree (3/5)

Useful options

Option	Purpose
`--file X`	identify a single file instead of a folder
`--samples N`	number of sample points (default 5)
`--points 10,30,50,70,90`	exact sample positions (percent or 0-1 fractions)
`--sample-len 12`	seconds of audio per sample
`--workers N`	number of parallel worker processes (default 1)
`--review-confidence 0.35`	flag for review below this confidence
`--min-agreement 0.5`	fraction of samples that must agree before trusting the winner
`--runtimes runtimes.json`	expected runtimes (minutes) for a sanity check
`--runtime-tolerance 4`	minutes of runtime difference tolerated
`-v`	per-sample logging

Runtime sanity check - supply a small JSON of expected minutes:

```
{ "S01E01": 74, "S01E03": 48, "S01E04": 48 }
```

```
python identify_dvd_episodes.py --dir ./rips --runtimes runtimes.json
```

Any match whose file duration is off by more than the tolerance is flagged (catches extended cuts and confidently-wrong matches).

Step 3 - Handle low-confidence matches

Anything printed under **“needs manual review”** (also `needs_review = true` in the CSV/JSON) is where to spend your attention. Common flags and what they mean:

- **`low confidence`** - the dialogue didn’t match strongly. Often a heavily re-encoded rip or a section with little dialogue; open the file and check the suggested episode, or re-run that one file with more samples: `--file X --samples 9`.

- `samples disagree (3/5)` - different parts of the file matched different episodes. Usually an **extended cut** or a file that contains **two episodes** (a DVD “play all” title). Split it or rename manually.
- `tie with S01Exx` - two episodes got equal votes; check both.
- `runtime ... vs expected` - likely an extended/edited cut, or a wrong match.

Tips to improve results:

- Increase samples for longer files: `--samples 9 --sample-len 15`.
- Make sure subtitles cover all the episodes you are trying to identify.
- Everything **not** flagged (`✓`) is safe to auto-rename.

Step 4 - Rename for Plex

Plex expects: `Show Name - SxxEyy - Optional Title.ext`, e.g. `Matlock (1986) - S01E04 - The Stripper.mkv`, inside `Matlock (1986)/Season 01/`.

Use the CSV to drive a rename. A minimal example (dry-run first!):

```
import csv, os, shutil

SHOW = "Matlock (1986)"
SRC  = "/path/to/dvd_rips"
DEST = "/plex/TV/Matlock (1986)"

with open("episode_map.csv", newline="", encoding="utf-8") as fh:
    for row in csv.DictReader(fh):
        if row["needs_review"].lower() == "true":
            print("SKIP (review):", row["filename"]); continue
        ext = os.path.splitext(row["filename"])[1]
        ep  = row["episode_id"]                       # e.g. S01E04
        season = ep[1:3]
        target_dir = os.path.join(DEST, f"Season {season}")
        os.makedirs(target_dir, exist_ok=True)
        new = f"{SHOW} - {ep}{ext}"
        print(f'{row["filename"]}  ->  Season {season}/{new}')
        # shutil.move(os.path.join(SRC, row["filename"]),
        #             os.path.join(target_dir, new))   # uncomment to apply
```

Review the printed plan, then uncomment the `shutil.move` line to apply. Files flagged for review are skipped so you can handle them by hand.

After moving, trigger a Plex library scan (**Plex -> library -> … -> Scan Library Files**), and Plex will match the episodes and pull artwork/metadata.

Quick reference

```
# 1. one-time: build reference DB from subtitles
python create_fingerprint.py --dir /path/to/subs

# 2. identify a whole folder of rips (parallel)
python identify_dvd_episodes.py --dir /path/to/dvd_rips \
    --csv episode_map.csv --json episode_map.json --workers 4

# 3. re-check a tricky file with more samples
python identify_dvd_episodes.py --file "/path/to/dvd_rips/Disc2_Title1.mkv" \
    --samples 9 --sample-len 15 -v

# 4. rename the confident ones for Plex (see script above)
```

> **Note on this VM:** the reference DB (`fingerprints.db`) and any test files live on the Abacus VM, not your local machine. Download them via the Files panel to run the tool locally against your own DVD rips.